Schema design for relational database

Presenter: itlinhph

ABOUT ME

Phan Hồng Lĩnh

- Technical Leader - GHTK (Performance & DBA)

fb.com/itlinhph

itlinhph@gmail.com

CONTENT

I. RELATIONAL DATABASE
II. SCHEMA DESIGN
III. TABLE DESIGN

I. RELATIONAL DATABASE

Data/Database:

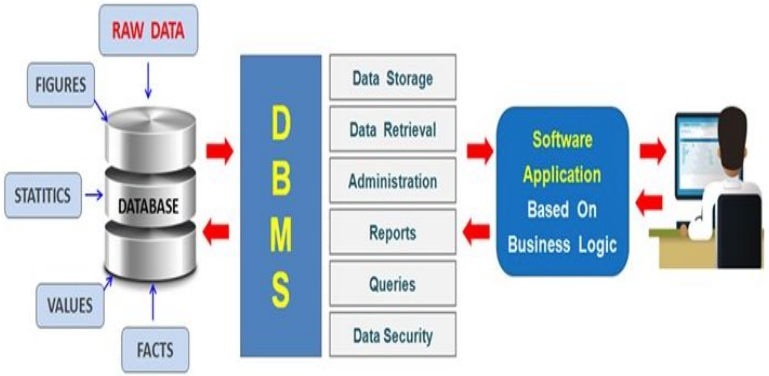
- A database - *systematic* collection of data.
- Database make data management easy.

Relational Database:

- Predefined relationships
- Constraint
- Tables, columns, rows

Relational Database Management System:

- MySQL, Oracle, SQL Server,...

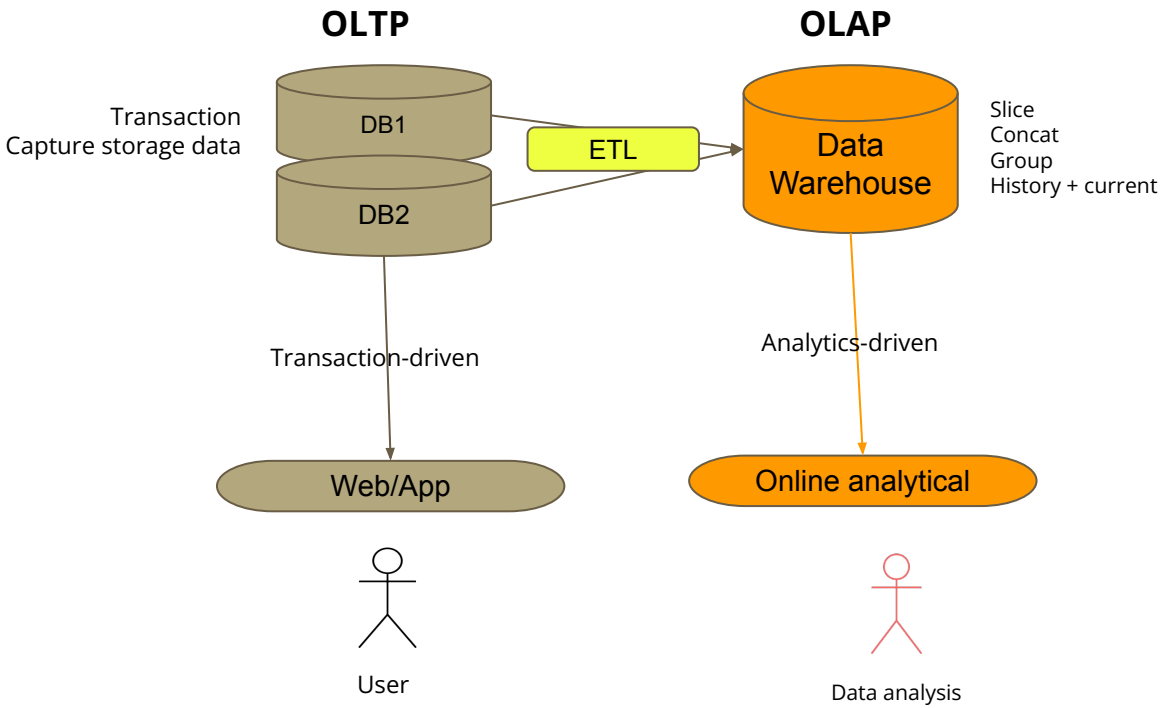


OLTP & OLAP

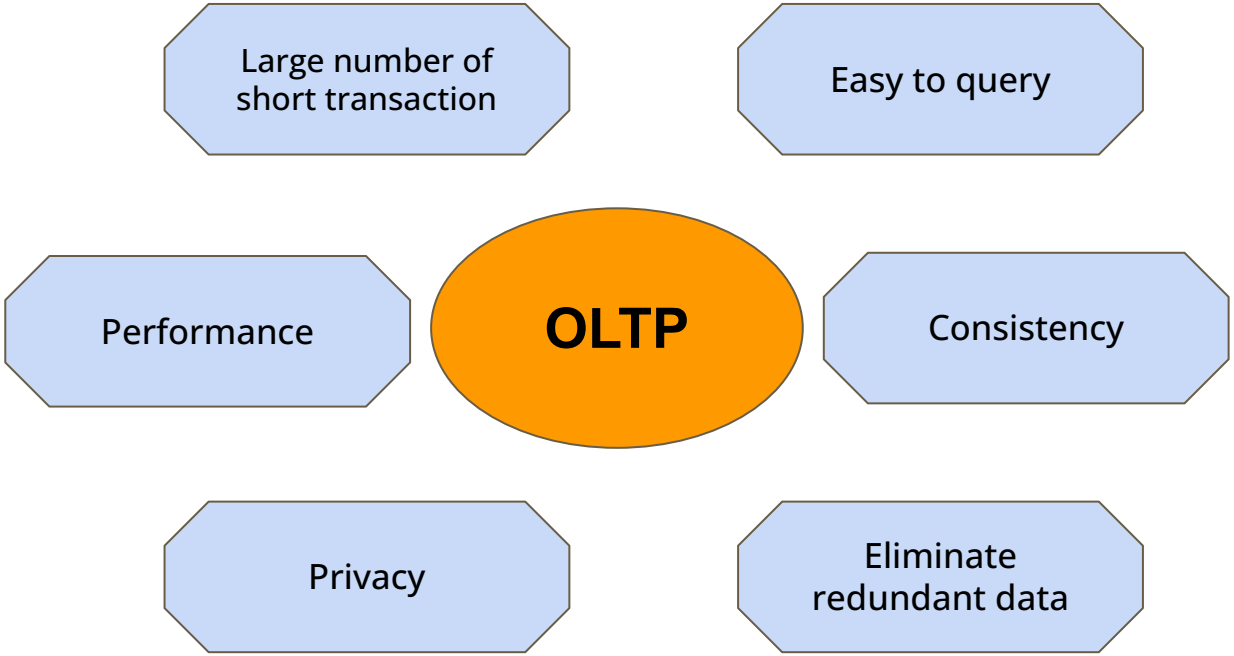
OLTP (*Online Transaction Processing System*)

OLAP (*Online Analytical Processing System*)

=> Focus on OLTP schema design



OLTP DATABASE DESIGN



OLTP SCHEMA DESIGN

Q: Why we need design a good schema?

A: "All roads lead to Rome" (Database)

Q: What is large database?

...

II. SCHEMA DESIGN

BASIC STEP

1. Collect and organize information
2. Find entities and attributes
3. Turning into Data Model
4. Apply the normalization rules
5. Design schema detail

COLLECT INFORMATION

- Objects, relationship and their attributes
- Range value of attributes
- Estimate data growth
- Hot & Cold data
- Frequency of CRUD
- ...

FIND ENTITIES AND ATTRIBUTES

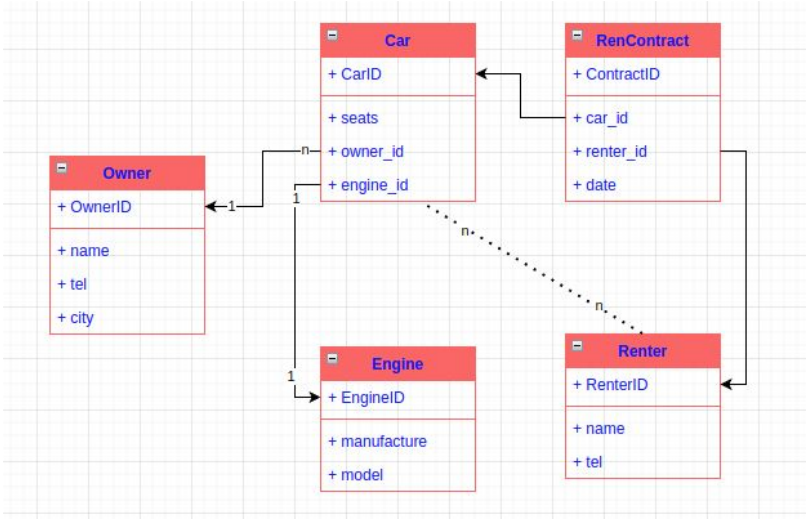
- Draft all objects and their attributes
- Should base on ***Class Diagram***, but NOT mapping 1-1

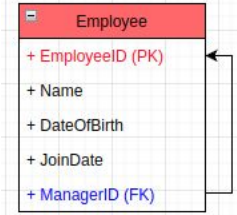
Class Diagram	Entity-Relationship Diagram
Class	Table
Attribute	Column
Attribute with collection	Columns, Foreign Key in another table
Object	Row
Object ID	Primary Key
Connection	Foreign key
Inheritance	Uniform primary key in multiple tables

TURNING INTO DATA MODEL

OBJECT RELATIONSHIPS:

- Association:
  - one-to-one: Create 2 tables for 2 classes, foreign key in a table
  - one-to-many: Create 2 tables for 2 classes, primary key in ***one*** is foreign key in ***many***
  - many-to-many: Create middle table
  - recursive: create a column with foreign key
- Aggregation, Composition: Consider like a Association connection
- Inheritance (to be continue...)

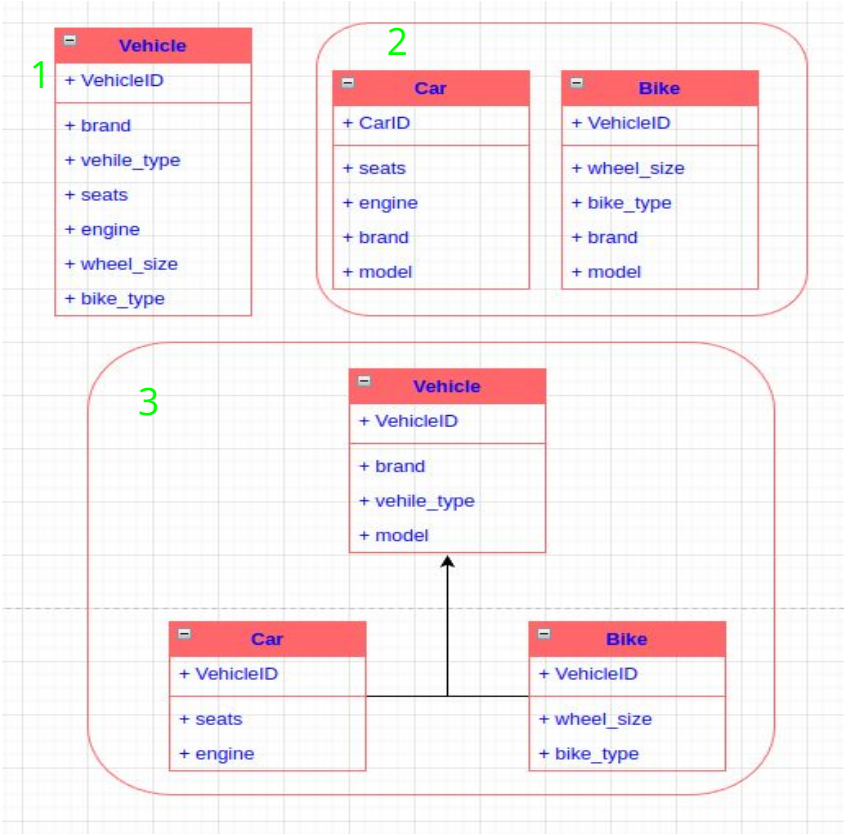




TURNING INTO DATA MODEL

OBJECT RELATIONSHIPS:

- Inheritance (B,C inheritance from V):
  1. Create only 1 table for V,B,C
  2. Create 2 tables for B,C (denormalization)
  3. Create 3 tables for V,B,C with the same ID

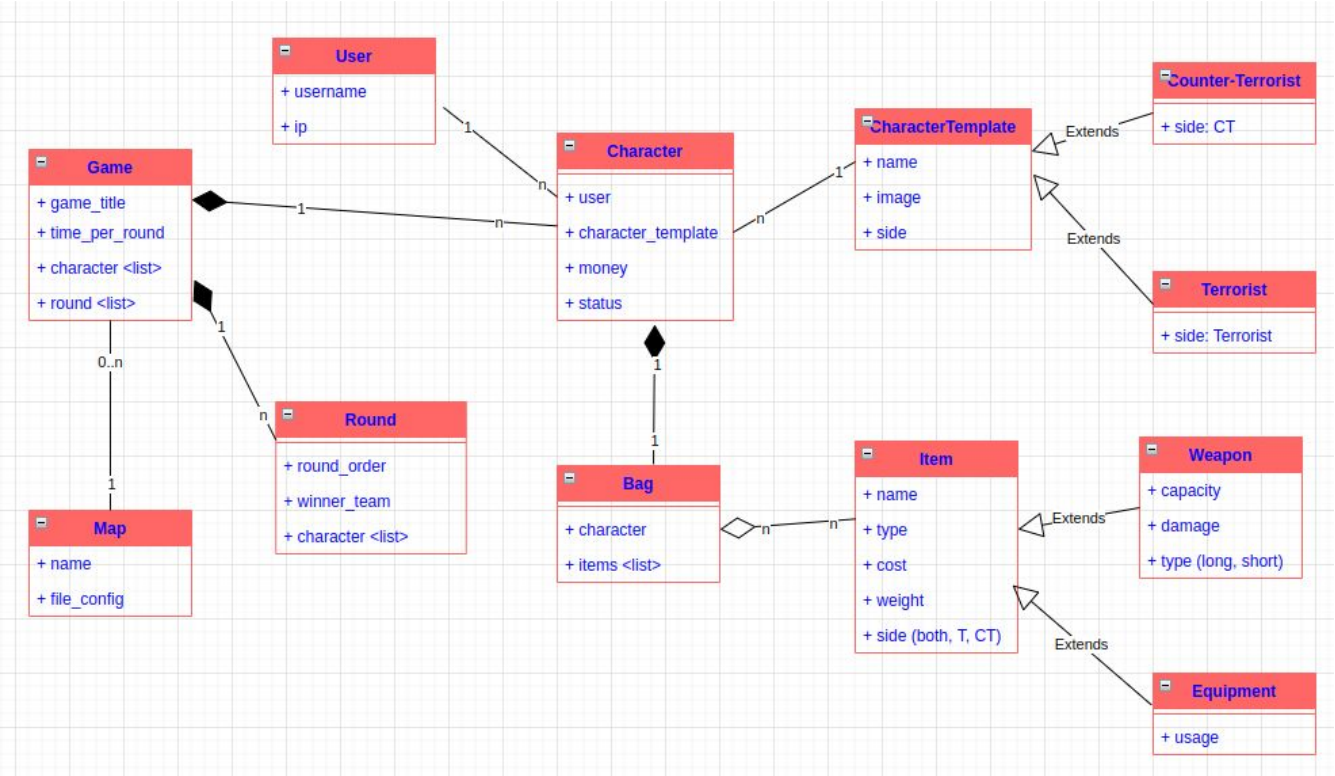


DESIGN COUNTER-STRIKE GAME

Find objects:

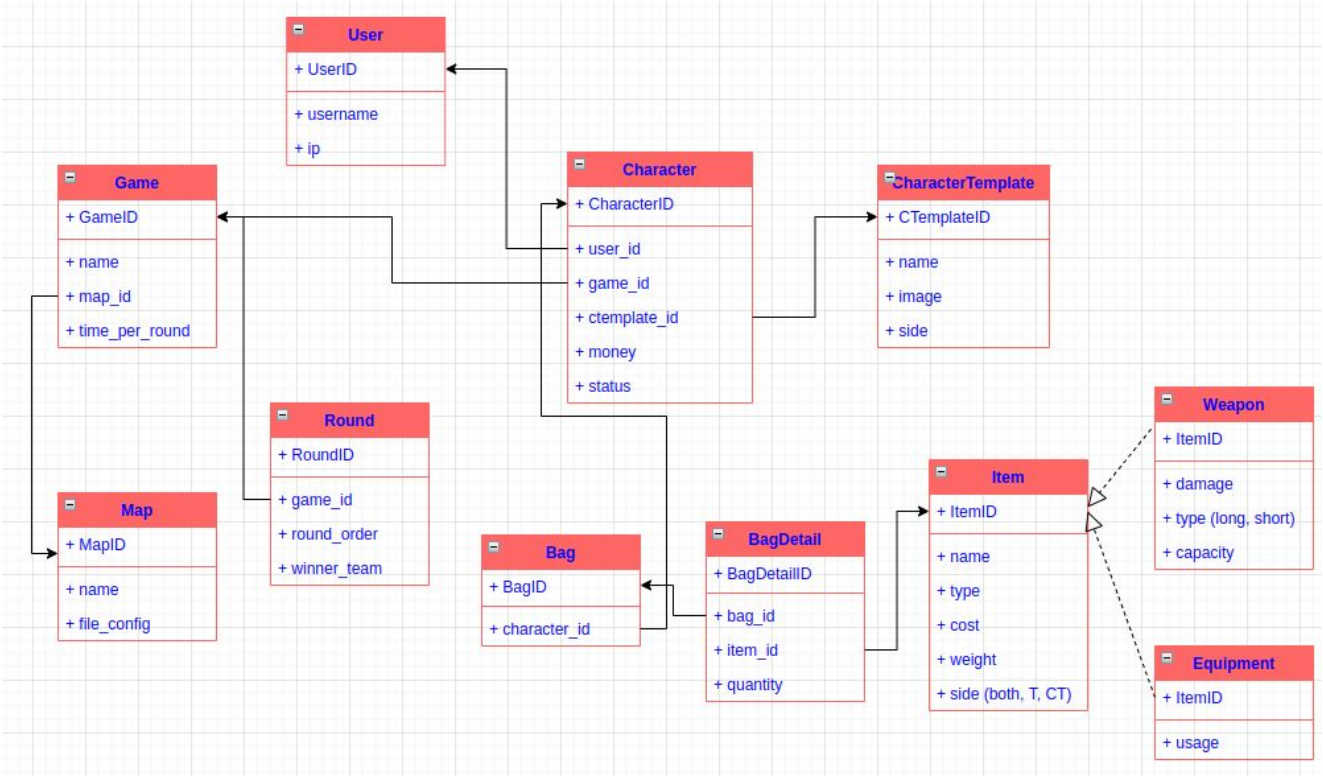
- Game, Map, Round
- Character: Counter-Terrorist, Terrorist
- Item: Weapon, Equipment
- Bag
- ...

DESIGN COUNTER-STRIKE GAME



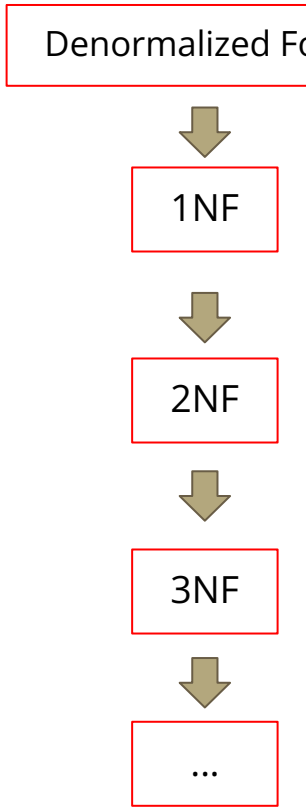
Class Diagram

DESIGN COUNTER-STRIKE GAME



Entity Diagram

NORMALIZATION RULES



- ➔ Every row and column have a single value, never have a list of values
- ➔ Value of each column belong to the same domain

- ➔ **Rule 1**: Be 1NF
- ➔ **Rule 2**: Every non-key column be fully dependent on the entire primary key, not on just part of the primary key.
  (If a table has a single column primary key, it automatically satisfies 2NF)

- ➔ **Rule 1**: Be 2NF
- ➔ **Rule 2:** Each non-key column must be dependent on the primary key and nothing but the primary key (Has no transitive functional dependencies)

NORMALIZATION: FEE SERVICES

PK

FromZone	ToZone	Transport	Transport Name	<0.5 (kg)	1.0 - 3.0	3.0-5.0	5.0-10.0	> 10.0
HN	HN	Road	Standard Express	10	15	20	20+(x-5)*6	55+(x-10)*4
HN	HCM	Road	Standard Express	13	19	30	24+(x-5)*6	60+(x-10)*5
HN	HCM	Fly	Urgent Express	19	21	27	21+(x-5)*7	53+(x-10)*6
HCM	BG,BN,HP	Road	Standard Express	9	13	18	14+(x-5)*8	48+(x-10)*7
BG,BN,HP	HN	Fly	Urgent Express	33	39	40	23+(x-5)*9	30+(x-10)*8
HN	BG,BN,HP	Road	Standard Express	21	27	45	50	50+(x-10)*9
BD,KG,CT	HN	Fly	Urgent Express	15	15	18	30+(x-5)*11	70+(x-10)*10

- ★ Zone: HN (Hà Nội), HCM (Hồ Chí Minh),
  BG, BN, HP (Bắc Giang, Bắc Ninh, Hải Phòng)
  BD,KG,CT (Bình Dương, Kiên Giang, Cần Thơ)
- ★ Fee currency: Thousand VND
- ★ x: Weight (kg)
- ★ 20+(x-5)*6 : 20k for base 5kg, add 6k foreach next 1kg

FEE SERVICES - 1NF

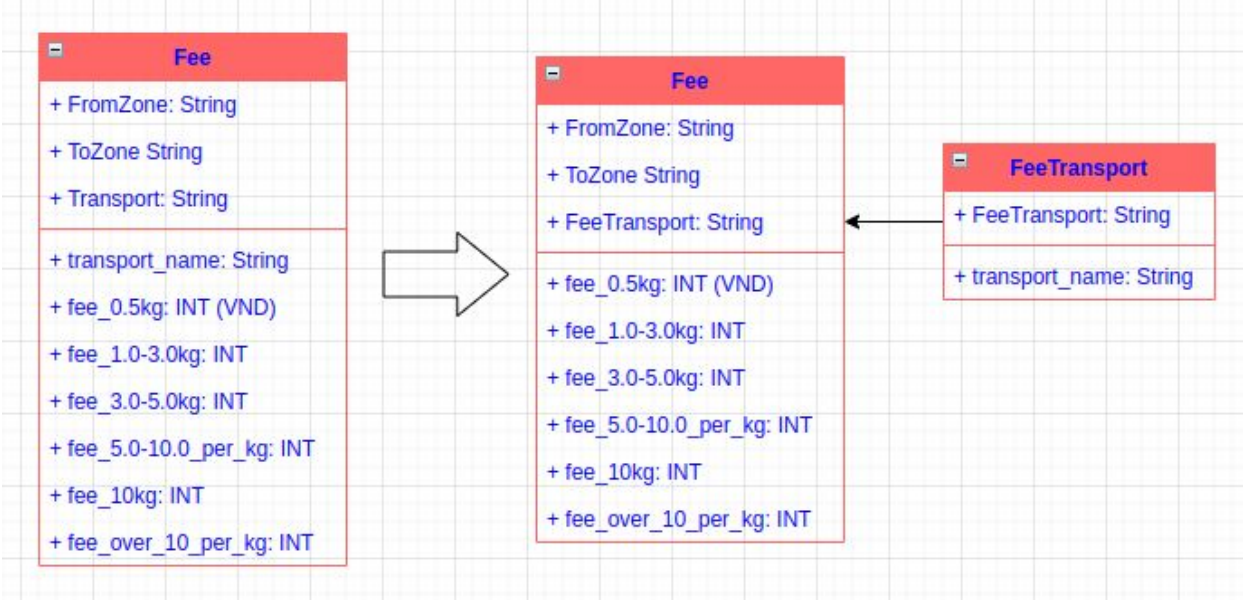
FromZone	ToZone	Transport	Transport Name	<0.5 (kg)	1.0 - 3.0	3.0-5.0	5.0-10 per kg	10.0	> 10.0 per kg
HN	BG	Road	Standard Express	21	27	45	0	50	10
HN	BN	Road	Standard Express	21	27	45	0	50	11
HN	HP	Road	Standard Express	21	27	45	0	50	12

- Remove list values
- All column has the same type of value

FEE SERVICES - 2NF

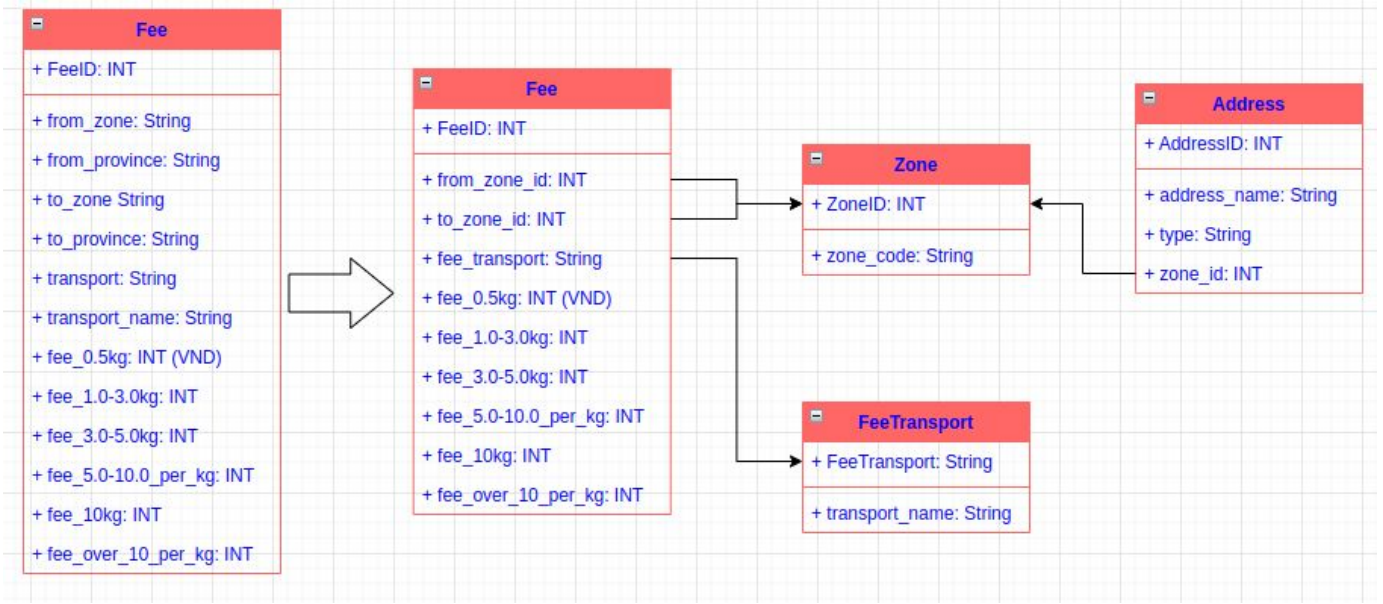
Service_name depend on mode (a part of primary key)

=> Divide into a table

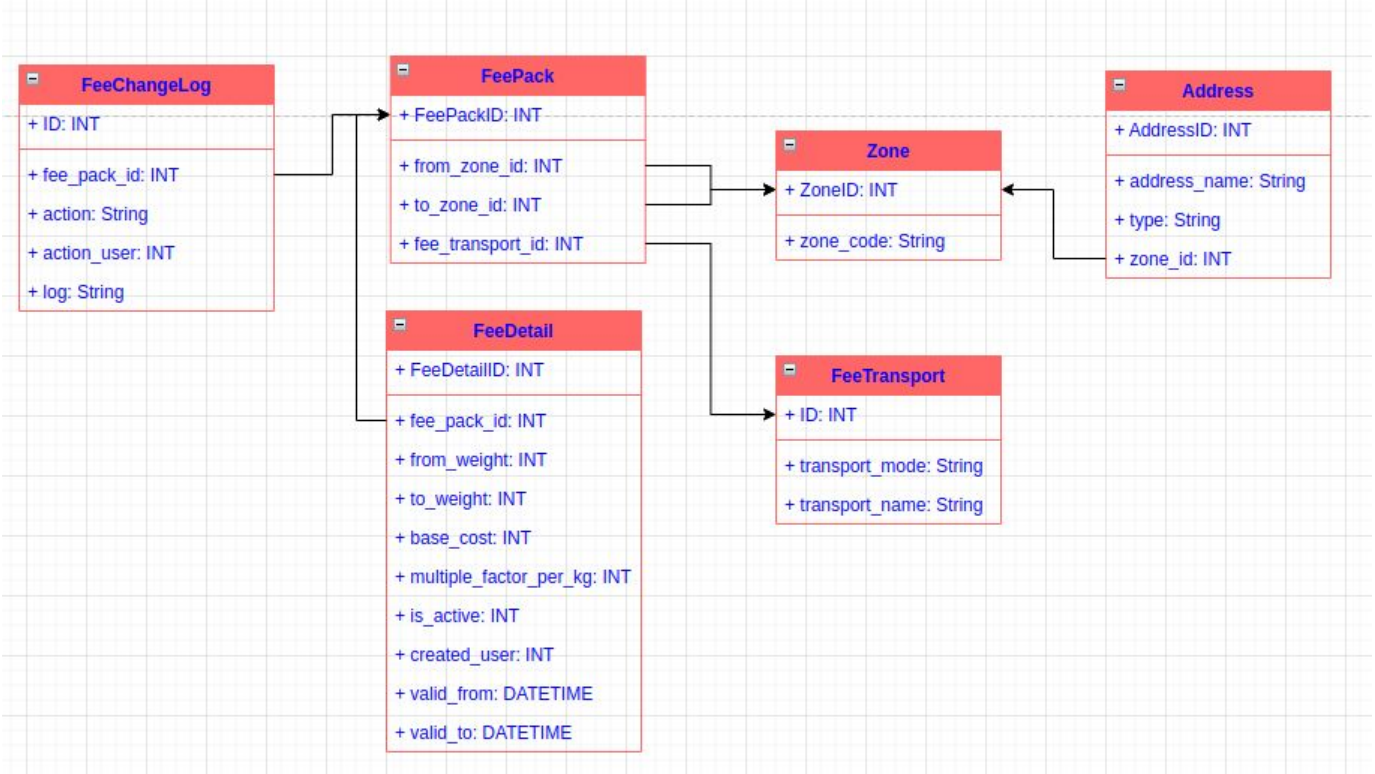


FEE SERVICES - 3NF

Remove transitive dependency



FEE SERVICES - SAFE & SCALABILITY



III. TABLE DESIGN

(Some terms focus on MySQL)

TABLE SCHEMA

- Primary Key
- Data Types
- Index & Partitions
- Constraints
- Charset & Collate

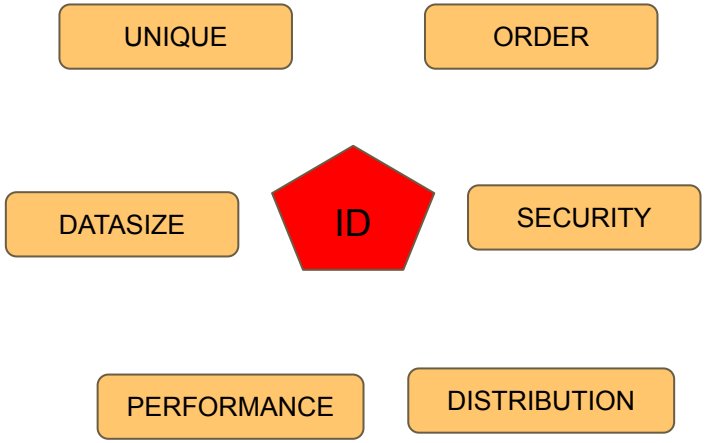
```
CREATE TABLE `fee_pack`
(
    `id`              int(11) unsigned      NOT NULL AUTO_INCREMENT,
    `name`            varchar(255)          DEFAULT '',
    `from_zone_id`    int(11) unsigned      NOT NULL COMMENT 'zone pickup',
    `to_zone_id`      int(11) unsigned      NOT NULL COMMENT 'zone deliver',
    `transport_mode`  enum ('road', 'fly') NOT NULL,
    `created`         datetime              DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (`id`),
    UNIQUE INDEX `IDX_1`(`from_zone_id`,`to_zone_id`,`transport_mode`)
) ENGINE = InnoDB
 DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;
```

PRIMARY KEY

- **AUTO INCREMENT**
  - + Simple to use, small data size, nextId
  - - Security (easy to guess), sharding, table locking
- **UUID**
  - + Simple to use, range of values, sharding
  - - Data size, cache, order, slow query, extra space
- **HASH**
  - + nextId, sharding, reduce data size from raw data
  - - Duplicate, order
- **SNOWFLAKE ID**
  - + Order, fast, distribution, security
  - - NodeID limited, sequence

=> Avoid string types for identifiers if possible

Primary key usually is clustered-index => Very importance in performance



DATA TYPES

★ **SMALLER IS USUALLY BETTER**

Use less space on the disk, in memory, CPU cache, fewer CPU cycles to process

★ **SIMPLE IS GOOD**

Fast to sorting, comparison

★ **AVOID NULL IF POSSIBLE**

Column has NULL value is harder to optimize queries (index statistics, value comparisons,...)

Use more storage space, complex logic

In some cases, should not avoid. EX: set default time “0000-00-00 00:00:00”

NUMBERS

INTEGER:

Choose smallest type as possible

Add *UNSIGNED* if possible

REAL NUMBERS:

Be careful with *FLOAT*

Data Type	Storage Required
TINYINT	1 byte
SMALLINT	2 bytes
MEDIUMINT	3 bytes
INT, INTEGER	4 bytes
BIGINT	8 bytes
FLOAT(*p*)	4-8 bytes
FLOAT	4 bytes
DOUBLE [PRECISION], REAL	8 bytes
DECIMAL(*M*, *D*)	Varies
BIT(*M*)	approximately (*M*+7)/8 bytes

STRING

VARCHAR vs CHAR

- *VARCHAR* use less data size but poor in compare than *CHAR*
- Always consider use *CHAR* with fixed-length data (except case only 1 char)
- *VARCHAR(n)*: n smaller is better in-memory

TEXT

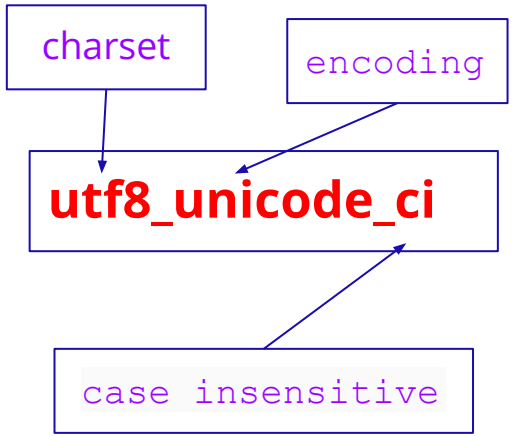
- Consider move *TEXT* value into another table/storage
- Compare *TEXT*: use hash value instead

ENUM vs STRING vs TINYINT

- Consider use *TINYINT* > *ENUM* > *STRING* for predefined data
- Beware of *ENUM* with constants value like *ENUM*('1', '2', '3')

STRING: CHARSET & COLLATE

- **CHARSET**: *ascii* vs *utf8* vs *utf8mb4*
  - *ascii* ≤ *utf8* ≤ *utf8mb4* in data size
  - *utf8mb4* usually slower than *ascii* (~50%)
- **ENCODING**: *bin* vs *general* vs *unicode*
  - *bin* > *general* > *unicode*: Fast comparison, sort operations
- **COLLATE**: set of rules to compare characters
  - When comparison (include join) : column should be the same character set and collation (avoid string conversions)



SPECIAL TYPES

DATE & TIME:

- TIMESTAMP: 4 bytes storage, from 1970 to '2038-01-19'
- DATETIME: 5-8 bytes, contain timezone, from '1000-01-01' to '9999-12-03'
- MySQL 5+, TIMESTAMP values are converted from the current time zone to UTC for storage, and converted back from UTC to the current time zone for retrieval.

JSON:

- Use when data schema isn't important.
- Spend more space to store JSON schema
- Query slower than traditional SQL columns

INDEX

TYPE OF INDEX:

- B-tree, Hash Index,...
- Unique index, single-column, multiple-column index,...

INDEXING STRATEGIES FOR HIGH PERFORMANCE:

- Prefix Indexes (indexing the first few characters instead of the whole value)
- Index Selectivity (choose column with the good cardinality)
- Multicolumn Indexes
- Choosing a Good Column Order
- Redundant and Duplicate Indexes, Unused Indexes
- Add NOT NULL if possible
- ...

PARTITIONS & CONSTRAINTS

PARTITIONS:

- Determine hot & warm data to choose a good partition key
- Easy to archives, query
- Disadvantages: may change schema, table locks, some feature not support (foreign key,...)

CONSTRAINTS:

- FOREIGN KEY: Usually disable to increase performance
- UNIQUE: Good for select query, but bad for insert/update

ANY QUESTION?